

More tuples

1. Returning several values from a function

Returning several values

When several values are returned, they are returned as a tuple

```
1 def split_in_half(word):
2     split_at = len(word)//2
3     first_half = word[:split_at]
4     second_half = word[split_at:]
5     return first_half, second_half # Returning two values
6
7 splitted = split_in_half("hello")
8 print(splitted)
9 print(type(splitted)) # A tuple is returned
10 print(splitted[0])
11 print(splitted[1])
12 print("-----")
13
14 # Each tuple elements instead directly assigned to variables
15 first, second = split_in_half("hello")
16 print(first)
17 print(second)
```

```
1 def split_in_half(word):
2     split_at = len(word)//2
3     first_half = word[:split_at]
4     second_half = word[split_at:]
5     return first_half, second_half # Returning two values
6
7 splitted = split_in_half("hello")
8 print(splitted)
9 print(type(splitted)) # A tuple is returned
10 print(splitted[0])
11 print(splitted[1])
12 print("-----")
13
14 # Each tuple elements instead directly assigned to variables
15 first, second = split_in_half("hello")
16 print(first)
17 print(second)
```

```
('he', 'llo')
<class 'tuple'>
he
llo
-----
he
llo
```

2. Fikonspråket

(another example of a function returning two values)

```
1 def get_index_first_vowel(word):
2     vowels = {"a" , "e" , "i" , "o" , "u" , "y"} #no Swedish
3     for i in range(0, len(word)):
4         if word[i] in vowels:
5             return i
6     return 0
7
8 def code_word(word):
9     index_first_vowel = get_index_first_vowel(word)
10    split_at = index_first_vowel + 1 # Split after first vowel
11    print(split_at)
12    first_word = "fi" + word[split_at:]
13    second_word = word[:split_at] + "kon"
14
15    return first_word , second_word
16
17 for word in ["stump", "kaka", "velociped"]:
18     coded = code_word(word)
19     print(coded)
```

('fimp', 'stukon')

('fika', 'kakon')

('filociped', 'vekon')

```
1 def get_index_first_vowel(word):
2     vowels = {"a" , "e" , "i" , "o" , "u" , "y"} #no Swedish
3     for i in range(0, len(word)):
4         if word[i] in vowels:
5             return i
6     return 0
7
8 def code_word(word):
9     index_first_vowel = get_index_first_vowel(word)
10    split_at = index_first_vowel + 1 # Split after first vowel
11    print(split_at)
12    first_word = "fi" + word[split_at:]
13    second_word = word[:split_at] + "kon"
14
15    return first_word , second_word
16
17 coded = code_word("stump")
18 print(coded)
19 print(type(coded))
20 print(coded[0])
21 print(coded[1])
```

3. Looping over a list of tuples

Looping over a list of tuples

Option 1: Let the loop variable contain the tuple

```
1 sentence = [("The", "DET"), ("sun", "NOUN"),
2             ("shines", "VERB"), ("today", "ADV")]
3
4 for tagged in sentence:
5     print(tagged)
6
7 print(type(tagged))
```

```
('The', 'DET')
('sun', 'NOUN')
('shines', 'VERB')
('today', 'ADV')
<class 'tuple'>
```

Looping over a list of tuples

Option 2: Assign each element in the tuple to a separate loop variable

```
1 sentence = [("The", "DET"), ("sun", "NOUN"),
2             ("shines", "VERB"), ("today", "ADV")]
3
4 for word, pos in sentence:
5     print(word)
6
7     print(type(word))
8     print(type(pos))
```

```
The
sun
shines
today
<class 'str'>
<class 'str'>
```

```
1 sentence = [("The", "DET"), ("sun", "NOUN"),
2             ("shines", "VERB"), ("today", "ADV")]
3
4 for tagged in sentence:
5     print(tagged)
6
7 print(type(tagged))
```

```
('The', 'DET')
('sun', 'NOUN')
('shines', 'VERB')
('today', 'ADV')
<class 'tuple'>
```

```
1 for word, pos in sentence:
2     print(word)
3
4 print(type(word))
5 print(type(pos))
```

```
The
sun
shines
today
<class 'str'>
<class 'str'>
```